

zkCoins

Robin Linus¹

2023

Source: gist.github.com/RobinLinus/d036511015caea5a28514259a1bab119

A payment system with strong privacy and scalability, combining a client-side validation protocol with validity proofs.

zkCoins is a novel blockchain design with strong privacy and scalability properties. It combines client-side validation with a zero-knowledge proof system. The chain is reduced to a minimum base layer to prevent double spending. Most of the verification complexity is moved off-chain and communicated directly between the individual sender and recipient of a transaction. There are very few global consensus rules, which makes block validation simple. Not even a global UTXO set is required.

In contrast to zk-rollups there is no data availability problem, and no sequencer is required to coordinate a global proof aggregation. The protocol can be implemented as an additional layer contained in Bitcoin's blockchain (similar to RGB or Taro) or as a standalone sidechain.

The throughput scales to hundreds of transactions per second without sacrificing decentralization.

Design Principles

The core design principle is to use the chain for what the chain is good for, which is an immutable ordering of commitments to prevent double-spending. This means two things:

1. Never write into the blockchain what can be communicated off-chain.
2. Avoid validation on the global layer that can be performed on the client-side.

Client-Side Validation (CSV) and Validity Proofs

Client-side validation protocols meet these design principles well. The verification of a token's history is performed only by the recipient of that token. Transaction data and the token history is communicated off-chain from the sender to the recipient, which minimizes the load on the global layer.

Historically, the fundamental problem of CSV protocols was that token histories grow quasi-exponentially. However, recent advancements in the field of validity proofs (e.g.

¹ZeroSync

STARKs) now allow to compress token histories to a negligible size. A sender can incrementally extend a token history proof to prove a transaction's validity to its recipient.

This leads to our core idea: Blockchains scale significantly better by reducing them to serve as a minimal base layer for such a zkCSV protocol. We can reduce a block to be a simple list of seals. A seal can be represented in about 64 bytes. The size of a seal is constant, regardless of the corresponding transaction's number of inputs and outputs. All TX data is communicated off-chain, except for the seal.

Accounts-based Model

Each user requires only a single seal at a time, if we use an accounts-like model instead of an UTXO-like model. With each account update users aggregate into their account history proof all the inputs which they received since their last account update.

Timechain Accumulator

Almost no global consensus rules are required. We can even get rid of the UTXO set. There is no global state required that grows linearly with the number of users. A constant amount of memory suffices.

Each user requires only a proof of non-inclusion for their own current seal. More precisely, non-inclusion since the block in which that seal was created by the previous seal. To compute their proof of non-inclusion, once a month, nodes sort all seals that occurred in blocks during that month. Then they compute a Merkle tree over the sorted set of seals. The resulting Merkle paths can be used as proofs of non-inclusion for other seals, because we can show that a particular seal is not at the position where it would be, if it was included in the sorted set.

The older the seal of a user the more proofs of non-inclusion they have to aggregate into their account history proof. However, aggregating one Merkle path per month is negligible effort.

Privacy

Adding zero-knowledge to account history proofs allows to obfuscate both the transaction amounts and the transaction graphs. This provides perfect privacy and censorship-resistance because eavesdropper cannot correlate seals or distinguish transactions in any way.

Throughput Analysis

Assuming the parameters of the Bitcoin blockchain, so a block size of 4 MB and a block time of 10 minutes, and 64 bytes per seal. Then this protocol allows for $4 \text{ MB} / 10 \text{ mins} / 64 \text{ bytes} / \text{TX} = 104 \text{ TXs/second}$.

The throughput scales further. Block verification is much faster than in Bitcoin because no such thing as witness verification is required for global consensus. Also there is no UTXO set and no initial block download required because new users do not need old block data. Therefore, the block

time can be decreased without increasing hardware requirements in comparison to Bitcoin. Only more network bandwidth is required. Already a two times faster block time allows to surpass the throughput of PayPal.

Limitations

- Transactions are interactive, so the recipient has to be online. However, there's only one-way interaction required.
- Backups are crucial. Losing your account's history proof leads to loss of funds.
- The sender needs sufficient computational resources to prove their account updates. Is that feasible on a phone?
- In general, proof systems are highly complex, they are novel technology, and implementations are not very mature yet.
- Having BTC on zkCoins requires some kind of ZKP verifier on Bitcoin. For example, BitVM.
- There is no global state, so no smart-contracting capabilities.